Smart Knowledge Management System: A Gemini API Based Web Platform for Unified Document Organization and Conversational Learning

Shah Prem Jigneshkumar
Dept. of Computer Engineering (B.Tech CE)
Silver Oak University
Ahmedabad, India
premshah12072005@gmail.com

Shah Jeet Mehulbhai
Dept. of Computer Engineering (B.Tech CE)
Silver Oak University
Ahmedabad, India
jeetyashshah@gmail.com

Shah Jash Priteshkumar
Dept. of Computer Engineering (B.Tech CE)
Silver Oak University
Ahmedabad, India
jashshah600@gmail.com

***Abstract*—Learners and small teams today keep their study and reference material spread across many unconnected places, PDFs in one folder, class notes in another app, scanned pages saved as images, and long articles bookmarked but never revisited. Reviewing this material later usually means opening several tools one at a time and reading everything manually. This paper presents the Smart Knowledge Management System, a web-based platform that lets a user group related material into notebooks, upload documents and images into a notebook, and then ask questions about that material in plain language. The system extracts readable text from each uploaded file, stores it alongside the file record, and, when the user asks a question or requests a summary, quiz, or set of flashcards, combines that stored text with the request into a single instruction that is sent to a hosted generative artificial-intelligence API. The response is shown to the user and saved as chat history for later reference. The design deliberately avoids heavier retrieval infrastructure so that every step, from upload to answer, remains easy to explain, build, and demonstrate within the scope of an undergraduate project. The system has been implemented as a working prototype and verified through module-level functional testing; a large-scale user study is left for future work.**

Keywords—Knowledge Management, Generative AI API, Web Application, Document Processing, MongoDB, Conversational Learning, Prompt Engineering, Educational Technology.

I. INTRODUCTION

Over the last few years, students and early-stage researchers have accumulated more digital study material than ever before, lecture PDFs, scanned handwritten notes, research articles, and reference images. This material rarely lives in one place. A student preparing for an examination might have textbook chapters saved on a laptop, question papers stored as images on a phone, and reference notes typed into a separate note-taking application. When it is time to revise, the student has to manually open each source, read through it, and write a fresh summary by hand. This process is repetitive, time-consuming, and does not scale well as the amount of material grows.

Consumer AI products such as Google AI Studio, NotebookLM, and ChatGPT have shown that a conversational interface built on top of a capable generative AI model can make working with documents considerably faster: a user can ask a direct question instead of reading an entire document end to end. However, these tools are either general-purpose assistants that are not organized around a user's personal collection of study material, or they are aimed at a broad consumer audience with a feature set and infrastructure far beyond what a single learner or a small student project actually needs.

This paper describes the design and implementation of the Smart Knowledge Management System, a lightweight web application built specifically to let one person organize their own learning material into notebooks and converse with that material using a hosted generative AI API. The guiding design principle throughout this project has been simplicity: every processing step, from reading an uploaded file to producing an answer, is implemented directly and can be explained in a single sentence, rather than depending on multi-stage retrieval pipelines that would be difficult to justify, build, and evaluate within the scope of a semester-long academic project.

It is worth being clear about what this project is, and is not, claiming. It does not claim to out-perform, or even to match every feature of, large consumer products built by dedicated engineering teams; instead, it claims that a small, well-scoped subset of what those products do, letting a user talk to their own uploaded material, is achievable with a modest, understandable stack and is genuinely useful in a student's day-to-day study routine. The system is best understood as a demonstration that this subset of functionality does not require the scale or infrastructure of a commercial product to be worth building and using.

The remainder of this paper is organized as follows. The Related Work section reviews related consumer AI tools and prior academic work on chatbot systems built around hosted generative AI APIs. The Problem Statement section states the problem this project addresses, and the Objectives section lists what the system sets out to achieve. The Proposed System section explains what distinguishes this work from a plain file-storage tool and from a generic AI chat assistant. The System Architecture and Methodology sections describe the architecture and end-to-end workflow, followed by Implementation Details, Database Design, and Generative AI API Integration. The Functional Features and User Interface Design sections describe what was built and how it is presented to the user. Security and Privacy, Ethical Considerations, Functional Testing, Results and Discussion, Advantages, Limitations and Future Scope, and the Conclusion complete the paper.

The main contributions of this work can be summarized as follows:

- A notebook-centred data model that organizes a user's own documents and the conversations held about them together, rather than as separate files and a separate, unaware chat history.

- A direct-prompting design that grounds every generative AI request in a user's own stored material without a separate indexing or retrieval subsystem, keeping the pipeline auditable end to end.
- A single, reusable prompt-construction procedure that unifies chat answers, summaries, key points, FAQs, quizzes, and flashcards behind one code path, differing only by instruction template.
- A fully working, functionally tested prototype implementation, together with an honest account of the scale at which its current design choices hold and where they would need to change.

II. Related Work

A number of consumer applications and academic prototypes have explored how a hosted generative AI API can be placed behind a conversational interface. Table I summarizes the widely used consumer tools that motivated this project, along with the gap that each one leaves for a personal, notebook-organized knowledge workspace.

TABLE I. Related Consumer AI Tools and Identified Gaps

System	Key Features	Gap for This Project
Google AI Studio	Conversational AI, prompt experimentation, multimodal input, model configuration	Aimed at AI experimentation rather than organizing a personal collection of study material
NotebookLM	Upload sources, source-grounded conversations, summaries and study guides	General consumer product; not built as a small, self-hosted academic prototype
ChatGPT	General-purpose conversational AI, content generation, document understanding	Not organized around notebooks tied to a user's own uploaded files
Traditional file-storage systems	File upload, folders, keyword search, access control	No conversational interaction or AI-generated summaries and study aids

Beyond consumer products, several recent studies describe web or mobile chatbots built as a thin application layer over a hosted generative AI API rather than a custom-trained model. A career-advice chatbot demonstrated that a Python-based backend, a lightweight database, and a single generative AI API are sufficient to build a self-hosted, multi-domain conversational assistant with user accounts and stored conversation history [1]. A separate study verified that a mobile chatbot could rely entirely on a hosted Gemini API key rather than training or hosting its own model, confirming that this approach is practical for student-built applications [2]. Work on chatbot personalization has shown that adapting responses to a user through context supplied in the request itself, rather than through model retraining, is an effective and lightweight strategy [3], which directly informed how this project constructs its prompts.

Full-stack implementations combining a JavaScript or Python web stack with a hosted generative AI API have also been documented as practical architectures for chatbot applications, following a frontend-to-backend-to-API pattern comparable to the one adopted in this project [4], [7]. Other studies describe chatbots that call a backend service to reach an external generative AI API securely, an approach this project follows by never exposing the API key to the browser [5]. Domain-specific assistants built on hosted generative AI APIs, covering areas from wellness support [6] to food and nutrition information [9], illustrate that a single general-purpose API can be adapted to a specific domain purely through prompt design, without any change to the underlying model. A comparative study of hosted conversational AI providers found that model choice is often driven by response speed, cost, and ease of integration rather than raw capability alone [8], which supports this project's choice of a single, well-documented API rather than integrating multiple providers. Finally, work on multi-turn assistant interfaces has highlighted the importance of maintaining a readable chat history for usability [10], a requirement reflected directly in the data design presented later in this paper.

Taken together, this body of work confirms that a thin, well-structured application layer around a single hosted generative AI API is a proven and academically documented architecture. What is missing from the consumer tools is a notebook-first organizational model built specifically for a student's own material, and what is missing from most of the smaller chatbot studies is that organizational layer combined with document-aware features such as summaries, quizzes, and flashcards generated directly from a user's own uploads. The Smart Knowledge Management System is positioned to close exactly this gap.

It is also worth noting a limitation shared by every system reviewed above, including this one: relying on a hosted generative AI API means depending on a third-party service's availability, response time, and pricing. A chatbot built this way is only as reliable as the API it calls, and none of the reviewed studies, nor this project, report on behavior when that external service is slow or temporarily unavailable. This is a shared characteristic of the whole hosted-API application category rather than a flaw specific to any one system, and it is revisited for this project specifically in the Limitations and Future Scope section.

III. Problem Statement

Based on the gaps identified above, this project addresses the following recurring difficulties faced by students and independent learners when working with digital study material:

- Study material is scattered across multiple applications and file types, with no single place to bring it together.
- Reviewing a document for revision requires reading it in full; there is no quick way to ask a direct question about its content.
- Creating summaries, question sets, or flashcards from a document is a manual and repetitive task.
- General-purpose AI assistants are not aware of a user's own uploaded material unless it is pasted in manually for every question.
- Existing document-management tools organize files but offer no conversational or content-generation features.

IV. Objectives

A. Primary Objectives

- Design and implement a web-based platform that lets a user organize uploaded material into notebooks.
- Support upload and text extraction for common formats: PDF, DOCX, TXT, and images.
- Provide a conversational interface that answers questions using the content of a selected notebook.
- Persist user accounts, notebooks, documents, and chat history in a structured database.

B. Secondary Objectives

- Generate summaries, key points, frequently asked questions, quizzes, and flashcards from uploaded material on request.
- Provide a simple prompt-playground view, similar in spirit to Google AI Studio, for experimenting directly with the generative AI API.
- Support keyword-based search across notebook titles, file names, and notes.
- Keep the implementation small enough to be fully understood, built, and demonstrated by a single student team within one semester.

V. Proposed System and Its Uniqueness

The Smart Knowledge Management System is organized around a single concept: the notebook. A notebook is a named collection that belongs to one user and can contain any number of uploaded documents together with the chat conversations held about them. This mirrors the mental model a student already uses when preparing for a subject, gathering all the relevant material for that subject in one place, rather than the flat file list used by conventional cloud-storage tools.

What distinguishes this system from the two obvious alternatives, a plain cloud-storage tool and a generic AI chat assistant, is the combination of three properties held together: persistent, per-notebook organization of source material; direct, automatic use of that material's extracted text whenever the user asks a question or requests a study aid inside that notebook, with no manual copy-pasting required; and generation of study products, summaries, key points, FAQs, quizzes, and flashcards, as first-class actions rather than something the user has to think to ask for in the right words. Table II summarizes this comparison.

TABLE II. Comparison of the Proposed System with Related Alternatives

Capability	Plain File Storage	Generic AI Chat	Proposed System
Organizes files by topic	Yes (folders)	No	Yes (notebooks)
Extracts and stores document text automatically	No	No	Yes
Answers questions using a user's own uploaded content, without manual copy-pasting	No	No	Yes
Generates summaries, quizzes and flashcards on request	No	Only if content is pasted in manually	Yes, built-in
Saves chat history tied to a specific notebook	Not applicable	General history only	Yes

The core modules of the proposed system are: user authentication, notebook management, document upload and text extraction, an AI chat interface scoped to a notebook, a set of one-click study-aid generators, a small prompt-playground for direct experimentation with the generative AI API, keyword search, and account settings. Each of these is described further in the sections that follow.

VI. System Architecture

The system follows a conventional three-tier web architecture, illustrated in Fig. 1. The presentation layer is a Next.js and React application written in TypeScript, styled with Tailwind CSS and the shadcn/ui component library. The application layer is a FastAPI service written in Python, which exposes a REST API, enforces authentication using JSON Web Tokens, and coordinates the document-processing and AI layers described below. The data layer is a MongoDB database that stores users, notebooks, document metadata and extracted text, and chat history; the original uploaded files themselves are kept in local file storage during development, with cloud object storage identified as a direct upgrade path for deployment.

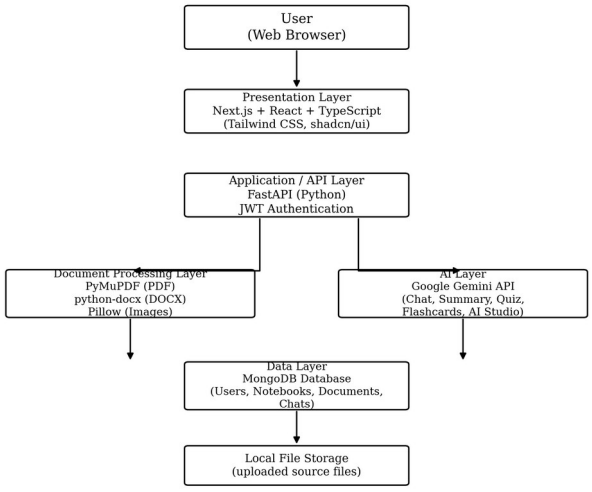


Fig. 1. Three-tier system architecture of the Smart Knowledge Management System.

A. Frontend Layer

The frontend is a Next.js application written in TypeScript, chosen because its component-based structure keeps the dashboard, notebook view, chat interface, and settings pages organized as independent, reusable pieces. Tailwind CSS handles layout and responsive styling, while

the shadcn/ui component library supplies ready-made buttons, dialogs, dropdowns, and form elements so that visual design effort could be spent on the notebook and chat experience rather than on rebuilding basic interface elements from scratch. All calls from the frontend to the backend go through a small API-client module built on Axios, which attaches the authentication token to every request and centralizes error handling.

B. Backend and Application Layer

The backend is a FastAPI service written in Python. FastAPI was selected over alternatives because it is quick to build REST endpoints in, validates request and response data automatically through Pydantic models, and produces interactive API documentation without extra effort, which was useful while the frontend and backend were being developed in parallel. This layer owns all business logic: verifying a submitted token before allowing access to a route, checking that a notebook belongs to the requesting user before returning its documents, and coordinating calls into the document-processing and AI layers described below.

C. Document-Processing Layer

This layer is responsible for turning an uploaded file into plain text. PyMuPDF is used for PDF files, python-docx for Word documents, and Pillow for validating and preparing image uploads before they are stored. Plain-text files are read directly. Each extractor is implemented as a small, independent function that takes a file path and returns text, which keeps the set of supported formats easy to extend by adding one new function rather than modifying shared logic.

D. AI Layer

The AI layer is a thin client around the Google Gemini API and is the only part of the system that communicates with an external generative-AI service; it is used by the chat interface, the summary and study-aid generators, and the prompt playground. Keeping this layer thin and clearly separated from the rest of the backend makes it straightforward to audit exactly what data is sent externally, and would make it possible to substitute a different hosted API in the future with minimal change elsewhere in the system.

E. Data and Storage Layer

MongoDB stores all structured application data, users, notebooks, document metadata, extracted text, and chat history, described in full in the Database Design section. Original uploaded files are kept in local file storage during development; migrating this to a cloud object store is identified as a direct and low-risk upgrade path for a future deployment, since the rest of the system only ever refers to a file by its stored path and would not need to change.

VII. METHODOLOGY

Fig. 2 shows the end-to-end workflow that the system follows from account creation through to receiving an AI-generated response. The steps are described below.

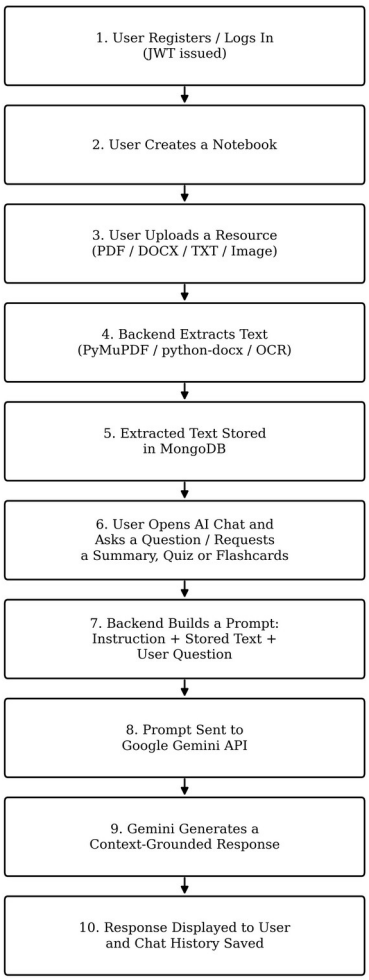


Fig. 2. End-to-end workflow of the Smart Knowledge Management System.

A user first registers or logs in, after which the backend issues a signed token that authorizes subsequent requests. The user then creates a notebook and uploads one or more resources into it. As soon as a file reaches the backend, the document-processing layer extracts its readable text, and both the file's metadata and the extracted text are stored in MongoDB under that notebook. When the user opens the AI chat for a notebook and asks a question, or presses one of the study-aid buttons, the backend retrieves the previously stored text for the documents in that notebook and combines it with a short instruction and the user's request into a single prompt. This prompt is sent to the Gemini API, and the returned response is displayed to the user and appended to that notebook's saved chat history so that the conversation can be revisited later.

This project deliberately builds the prompt by directly including the relevant extracted text rather than adding a separate indexing or retrieval stage. For the scale this system targets, individual notebooks made up of a handful of documents such as lecture notes, a research paper, or a set of scanned pages, the combined text comfortably fits within a single request to the generative AI API. This keeps the data flow easy to trace from upload to answer, keeps the amount of infrastructure code small enough for a student project to build and maintain, and avoids introducing components whose behavior would be hard to explain or verify at this project's scale. The Results and Discussion section revisits

the trade-off this choice makes as the size of a notebook grows.

VIII. Implementation Details

A. Prompt Construction Procedure

Every AI-facing feature in the system builds its request to the generative AI API using the same underlying procedure, listed below, with only the instruction text changing between features.

- Look up the notebook the request belongs to and confirm it belongs to the requesting user.
- Read the previously stored extracted text for every document in that notebook.
- Select the instruction template for the requested feature (chat answer, short summary, detailed summary, key points, FAQ, quiz, or flashcards).
- Combine the instruction, the stored text, and, for the chat feature, the user's question, into a single prompt string.
- Send the prompt to the Gemini API and wait for the generated response.
- Return the response to the frontend and, for the chat feature, append the question and answer to that notebook's chat history.

Because this procedure is identical across features, the backend implements it once as a shared function and passes in only the instruction template that differs per feature, which keeps the codebase small and avoids duplicated request-handling logic.

B. Backend API Endpoints

The backend exposes a small, purpose-built set of REST endpoints, summarized in Table III. Every endpoint other than registration and login requires a valid authentication token, and every notebook-scoped endpoint additionally checks that the requested notebook belongs to the caller before proceeding.

TABLE III. Summary of Backend API Endpoints

Endpoint	Method	Purpose
/auth/register	POST	Create a new user account
/auth/login	POST	Authenticate a user and issue a token
/notebooks	GET, POST	List a user's notebooks or create a new one
/notebooks/{id}	PATCH, DELETE	Rename or delete a notebook
/notebooks/{id}/documents	POST	Upload a file and extract its text
/notebooks/{id}/chat	POST	Ask a question or request a study aid
/notebooks/{id}/chat/history	GET	Retrieve saved chat history for a notebook
/search	GET	Search notebooks, file names, and notes by keyword

IX. Database Design

MongoDB was chosen for its flexible, document-oriented structure, which matches the naturally nested relationship between a user, their notebooks, and the documents and chat messages inside each notebook. Table IV summarizes the four main collections used by the system.

TABLE IV. Core Database Collections

Collection	Key Fields	Purpose
Users	user id, name, email, hashed password	Stores account credentials and profile information
Notebooks	notebook id, user id, title, created date	Groups documents and chats belonging to one topic
Documents	document id, notebook id, file name, file type, extracted text, upload date	Stores metadata and extracted text for each uploaded file
Chats	chat id, notebook id, question, answer, timestamp	Stores conversation history for each notebook

Passwords are never stored in plain text; they are hashed using bcrypt before being written to the Users collection. Storing the extracted text alongside each document, rather than re-reading the original file on every request, means that a chat request only needs a small number of database reads before a prompt can be built and sent to the generative AI API.

X. Technology Selection and Justification

Every major tool in the stack was chosen after weighing it against at least one alternative that the team considered. This section records those decisions so that the reasoning, not just the final list of technologies, is available to anyone reviewing or extending this project.

A. Next.js and React over a plain single-page setup

A plain React application built with a bundler alone would have worked, but Next.js was chosen because it provides file-based routing and a conventional project structure out of the box, which reduced the amount of setup decision-making needed before feature work could begin. TypeScript was layered on top for the same reason it is popular in industry: catching mismatched data shapes between the frontend and the FastAPI backend at compile time rather than at runtime.

B. FastAPI over Node.js and Express

Both FastAPI and an Express-based Node.js backend were considered, since either would be a reasonable partner for a React frontend. FastAPI was preferred because the team's document-processing needs, PDF and DOCX text extraction, and the official Gemini SDK, are all Python-first, so writing the backend in Python avoided maintaining two different language ecosystems for what is otherwise one connected pipeline. FastAPI's automatic request validation was an additional, if secondary, benefit during development.

C. MongoDB over a relational database

A relational database such as PostgreSQL would also have modeled this data correctly, but MongoDB was chosen because the natural relationship in this system, a user containing notebooks, each containing documents and chats, maps directly onto nested documents without the join queries a relational schema would need for the same views. For a system of this size, this reduced both the number of database queries per request and the amount of schema-migration code that had to be written and maintained.

D. A single hosted generative AI API over multiple providers

The team also considered integrating more than one generative AI provider so that a user could pick between them, a pattern seen in some of the reviewed literature [8]. This was deliberately not done: supporting multiple providers would have meant writing and maintaining a separate prompt-formatting and response-parsing path for each one, for a benefit, provider choice, that was judged to matter far less to a student user than having every feature work reliably against one well-documented API. The thin AI-layer design described earlier in this paper would make adding a second provider possible later without a major rewrite, should that trade-off change.

E. Cost and Resource Feasibility

A guiding constraint throughout technology selection was that the whole system had to be buildable and runnable by a student team without paid infrastructure. Every core tool in the stack, Next.js, FastAPI, MongoDB (run locally or on a free-tier cloud instance), and the free usage tier of the Gemini API, can be run at no direct cost during development and small-scale demonstration. This ruled out any approach that would have required training or hosting a custom model, which would have needed both a labelled dataset the project does not have and computing resources well beyond a typical student budget. Relying on a hosted generative AI API instead trades a small per-request cost at a larger future scale for zero upfront training cost, which is the right trade-off for a project at this stage.

XI. Generative AI API Integration

All AI-facing features in this system are implemented through the Google Gemini API, accessed from the FastAPI backend using the official Python SDK. The API key is stored only on the server and is never sent to the browser, so all requests to the generative AI service pass through the backend, which also logs and rate limits usage.

Each feature in the system is implemented as a different instruction template applied to the same underlying call. For the chat feature, the instruction asks the model to answer the user's question using only the supplied notebook text. For the summary feature, the instruction asks for a short or detailed summary of the same text. The key-points, FAQ, quiz, and flashcard generators follow the same pattern with an instruction specific to that output format. Because every feature reduces to "instruction plus stored text plus optional user question", adding a new study-aid type in the future is a matter of writing a new instruction template rather than changing the underlying data flow.

A separate prompt-playground view, inspired by Google AI Studio, lets a user type an arbitrary prompt and choose basic generation settings, including which Gemini model variant to use, the temperature, and a maximum response length, before viewing the raw response. This view is not tied to a notebook and is intended for direct experimentation with the API rather than for day-to-day notebook use.

A. Illustrative Interaction Walkthrough

To make the data flow concrete, consider a student who has uploaded a ten-page lecture PDF on data structures into a notebook titled "DSA Midterm" and then types the question "What is the time complexity of binary search, and why?" into that notebook's chat box. The backend confirms the notebook belongs to the student, reads the previously extracted text of the uploaded PDF, and builds a prompt equivalent in spirit to: "Using only the following study material, answer the student's question. If the material does not contain the answer, say so explicitly. Study material: [extracted PDF text]. Question: What is the time complexity of binary search, and why?" This prompt is sent to the Gemini API, and the response it returns is shown in the chat panel and appended to the notebook's saved history. If the same student instead pressed the "Quiz" button, the same extracted text would be combined with an instruction asking for a short multiple-choice quiz instead of a direct answer, following the identical procedure described above.

Table V summarizes the instruction template used for each feature at a conceptual level; the exact wording used in the implementation may be refined further, but the underlying pattern, instruction plus stored text plus optional user input, remains the same for every row.

TABLE V. CONCEPTUAL INSTRUCTION TEMPLATES BY FEATURE

Feature	Instruction Template (Conceptual)
Chat	Answer the question using only the supplied study material; say so if the answer is not present.
Short summary	Summarize the supplied study material in a short paragraph.
Detailed summary	Summarize the supplied study material in full detail, covering every major point.
Key points	List the key points of the supplied study material as short bullet points.
FAQ	Generate a list of frequently asked questions and answers based on the supplied study material.
Quiz	Generate a short multiple-choice quiz based on the supplied study material.
Flashcards	Generate a set of question-and-answer flashcards based on the supplied study material.

XII. FUNCTIONAL FEATURES

This section describes each implemented feature in more detail than the architectural overview above, grouped by the part of the system each one belongs to.

A. Account and Notebook Management

A user registers with a name, email address, and password, and logs in to receive an authentication token used for every subsequent request. From the dashboard, a user can create a notebook by giving it a title, rename or delete an existing notebook, and search across notebook titles. Each notebook belongs to exactly one user and is never visible to anyone else.

B. Document Upload and Extraction

Inside a notebook, a user can upload a PDF, DOCX, TXT, or image file. On upload, the backend immediately extracts the file's readable text using the appropriate library described earlier and stores both the file's metadata and its extracted text. Uploaded files can be listed, opened, or removed from within the notebook view.

C. AI Chat and Study-Aid Generation

The chat panel accepts a free-text question about the notebook's content and returns an answer grounded in the stored text, as described in the walkthrough above. Alongside free-text questions, one-click buttons generate a short summary, a detailed summary, a bulleted list of key points, a set of frequently asked questions, a short quiz, or a set of flashcards, each following the same underlying procedure with a different instruction.

D. Prompt Playground

Separately from any notebook, the playground screen offers a plain text box for an arbitrary prompt together with basic generation settings, model variant, temperature, and maximum response length, for direct, unstructured experimentation with the generative AI API.

E. Search and Account Settings

A keyword search bar matches against notebook titles, uploaded file names, and any notes attached to a notebook. The settings screen provides a dark-mode toggle, basic profile editing, and the logout action.

XIII. USER INTERFACE DESIGN

The interface was kept deliberately small: five screens cover every feature described above, so a first-time user can learn the whole system in a single sitting.

A. Dashboard

The dashboard is the first screen after login. It lists the user's most recently opened notebooks as cards, shows a short activity summary (recent chats and total documents uploaded), and provides a single button to create a new notebook, so that returning to ongoing work never takes more than one click.

B. Notebook and Chat View

Opening a notebook shows two panels side by side: a document list on the left, where files can be uploaded, renamed, or removed, and a chat panel on the right, where the user types a question or presses one of the study-aid buttons (summary, key points, FAQ, quiz, or flashcards). Every answer appears in the chat panel immediately below the question that produced it, and the full conversation remains available the next time the notebook is opened.

C. Prompt Playground

The playground is a separate, notebook-independent screen modeled loosely on Google AI Studio. It offers a plain text box for an arbitrary prompt, a choice of Gemini model variant, and sliders for temperature and maximum response length, with the raw generated response shown below. This screen exists purely for direct experimentation with the generative AI API and is not connected to any notebook's stored documents.

D. Search and Settings

A search bar available from any screen matches against notebook titles, file names, and notes and returns results grouped by type. The settings screen holds a dark-mode toggle, basic profile fields, and the logout action.

Across all five screens, layout follows a responsive design so that the same interface remains usable on a laptop or a tablet-sized browser window. A dark-mode option was included specifically because study sessions with dense reading material tend to run long, and reducing screen glare during extended use was judged a worthwhile, low-effort addition alongside the core features.

XIV. SECURITY AND PRIVACY CONSIDERATIONS

User passwords are hashed with bcrypt before storage, and every protected API route requires a valid JSON Web Token issued at login, so a request cannot read or modify another user's notebooks. Uploaded files are validated for type and size before processing to reduce the risk of malformed input reaching the document-processing layer. Because the generative AI API key is held only on the backend, it is never exposed to client-side code. At the current prototype stage, uploaded files are stored on the local file system rather than encrypted cloud storage, and there is no formal access-logging or audit trail; both are noted honestly as limitations in the Limitations and Future Scope section rather than presented as solved.

XV. ETHICAL CONSIDERATIONS

Because this system stores a user's uploaded study material and sends parts of it to an external generative AI service in order to answer questions, two ethical points

deserve explicit mention rather than being left implicit. First, a user should reasonably expect that anything they upload is used only to answer their own questions about it; the system does not use one user's uploaded material to answer another user's questions, and no uploaded content is used for any purpose beyond generating the response the user directly requested. Second, uploading copyrighted material, such as a purchased textbook, into the system and asking the generative AI API to summarize or quiz on it is subject to the same fair-use and copyright norms that would apply to summarizing that material by hand; the system does not police this automatically, and responsible use is left to the user, consistent with how comparable consumer tools such as NotebookLM operate.

A further consideration is over-reliance: a student who only ever reads AI-generated summaries and never the original material risks a shallower understanding than one who uses the system's answers as a starting point for further reading. This system is intended, and was designed, as a study aid that sits alongside a student's own reading, not as a replacement for it, and this framing is reflected in the chat interface always keeping the original uploaded document available and open next to the generated answer.

Finally, on data retention, a user's notebooks, documents, and chat history remain in the database until the user deletes them; the current prototype supports deleting individual notebooks, which removes their associated documents and chat history, but does not yet offer a single "delete my account and all my data" action. Adding one is a small, well-understood change and is listed alongside the other future-work items in the next section.

XVI. Functional Testing

Because this is an academic prototype rather than a production deployment, evaluation was carried out as module-level functional (black-box) testing: each feature was exercised with representative inputs and its output was checked against the expected behavior described in the preceding sections. No quantitative accuracy benchmark or user study was conducted; this is stated explicitly here and revisited as future work in the Limitations and Future Scope section, rather than reported as a numerical result that was not actually measured. Table VI summarizes the modules tested and the observed outcome for each. Testing was performed manually against the running backend using Postman for direct API calls and the deployed frontend for end-to-end checks, rather than through an automated test suite, which is itself noted as an area for future improvement.

TABLE VI. Module-Wise Functional Test Summary

Module	Test Performed	Observed Outcome
Authentication	Register, login with correct and incorrect credentials, access a protected route without a token	Behaved as expected in all cases
Notebook management	Create, rename, delete, and search notebooks	Behaved as expected
Document upload	Upload a PDF, a DOCX file, a TXT file, and an image; confirm text is extracted and stored	Text extracted and stored correctly for all supported formats
AI chat	Ask questions covering content present and absent in the uploaded document	Answered correctly grounded questions; declined to answer when content was absent
Study-aid generation	Request a summary, key points, an FAQ set, a quiz, and flashcards for the same document	Each generator produced output matching its intended format
Search	Search by notebook title and by file name	Returned matching records correctly

XVII. Results and Discussion

The functional testing summarized in Table VI confirms that the system behaves as designed for its intended use case: a single user working with a small number of notebooks, each holding a modest number of documents. The decision to build prompts by directly including a notebook's stored text, rather than adding a separate indexing or retrieval stage, worked well at this scale and kept every part of the pipeline, from an uploaded file to a generated answer, simple enough to trace and explain during a project review.

This same design choice is also the system's main known trade-off. As the total amount of text stored in a single notebook grows well beyond a handful of documents, the size of the combined prompt sent to the generative AI API grows with it, which would eventually increase response latency and API cost, and could approach the input-size limits of the underlying model. This is an accepted and disclosed limitation of the current design rather than an oversight, and the next section outlines how a future version could address it.

It is also worth being explicit about what this evaluation does and does not show. The functional tests in Table VI confirm that each module produces output of the expected type and shape; they do not measure how accurate, complete, or well-written that output is compared with a human reviewer, since doing so would require a structured study with multiple participants and a scoring rubric, which was outside the scope of this project. Similarly, all testing was performed by the development team on a small, hand-picked set of documents rather than a large or adversarial dataset, so the results describe correct behavior on realistic, well-formed input rather than robustness under stress. These boundaries are stated here so that the results are read for what they are: evidence that the system works as designed, not a measured claim about answer quality.

Table VII revisits the objectives set out earlier in this paper and states, for each one, whether it was met by the implemented prototype.

TABLE VII. Objectives Versus Implemented Status

Objective	Met By	Status
Organize uploaded material into notebooks	Notebook management module	Achieved
Support PDF, DOCX, TXT, and image upload with extraction	Document-processing layer	Achieved
Answer questions grounded in notebook content	AI chat feature	Achieved

Objective	Met By	Status
Persist accounts, notebooks, documents, and chat history	MongoDB data layer	Achieved
Generate summaries, key points, FAQs, quizzes, flashcards	Study-aid generators	Achieved
Provide an AI Studio-style prompt playground	Prompt playground screen	Achieved
Support keyword search	Search feature	Achieved (keyword-level only)
Integrate YouTube-link and audio ingestion	Not implemented in this version	Deferred to future work

XVIII. Advantages of the Proposed System

- Brings scattered study material into one organized, searchable place.
- Lets a user ask direct questions about their own material instead of re-reading it in full.
- Automates the creation of summaries, quizzes, and flashcards that would otherwise be made by hand.
- Uses a single, well-documented hosted API rather than training or hosting a custom model, keeping cost and complexity low.
- A simple, auditable data flow that is easy for reviewers, and for the developers themselves, to understand and verify.

Taken together, these advantages point to the same underlying property: every part of the system, from what data is stored to what is sent externally and why, can be explained in plain language to a reviewer, a classmate, or a future maintainer without reference to a black-box component. For a student project, where the ability to explain and defend design decisions matters as much as the decisions themselves, this auditability is as much a result of this work as any single feature.

XIX. Limitations and Future Scope

The current prototype has several limitations that are stated openly here rather than left unaddressed. Search is limited to keyword matching and does not understand the meaning of a query. Very large notebooks would eventually strain the direct-prompting approach described in the Methodology section. Uploaded files are stored locally rather than in encrypted cloud storage, and no formal user study has measured answer quality or user satisfaction. YouTube-link and audio ingestion, mentioned as a goal in early planning, was not implemented in this version of the system. Finally, as noted earlier in this paper, the system depends entirely on the availability, response time, and continued pricing of the external Gemini API; no fallback behavior is currently implemented for the case where that service is slow or temporarily unreachable.

Planned future work includes migrating file storage to a cloud object store for durability and scalability; adding transcript-based ingestion for YouTube links and audio recordings; conducting a structured user study to measure answer quality, response time, and overall satisfaction; and, if a future notebook regularly holds a large number of long documents, revisiting the direct-prompting design in favor of a more scalable way of selecting only the most relevant portions of text for a given question. Additional directions include a mobile-friendly interface, collaborative multi-user notebooks, a single account-and-data deletion action, and offline access to previously generated summaries and flashcards.

XX. Conclusion

This paper presented the Smart Knowledge Management System, a notebook-organized web platform that lets a user bring their scattered study material together and converse with it using a hosted generative AI API. By combining a conventional three-tier web architecture with a deliberately simple, direct-prompting approach to using the generative AI API, the system meets its stated objectives, organizing material, extracting and storing readable text, answering grounded questions, and generating summaries, quizzes, and flashcards, while remaining small enough for a single student team to build, understand, and demonstrate within one semester. Functional testing confirmed that each implemented module behaves as intended. The honestly reported limitations around search, scale, and the absence of a formal user study define a clear and realistic path for future work on this project.

More broadly, this project suggests that meaningful, personally useful AI applications do not require the heaviest available infrastructure to be worth building. For a student, or for a small team solving a well-scoped problem, a single hosted generative AI API combined with a carefully designed, notebook-first data model can deliver much of the day-to-day value of larger consumer AI products, while remaining small, explainable, and fully owned by the people who built it.

Acknowledgment

The authors thank the Department of Computer Engineering, School of Technology, Design and Computer Application, Silver Oak University, Ahmedabad, for the guidance and resources provided during the development of this project.

References

References [1]-[10] correspond to related chatbot and generative-AI-API application studies identified during the literature survey for this project. Titles are reproduced as supplied; readers intending to cite this work further are advised to confirm complete bibliographic details (authors, venue, and year) for these items before formal citation.

[1] “Domain Pro Chatbot: A Multi-Domain AI Career Advisor System.”

[2] “Implementasi LLM Gemini Pada Pengembangan Aplikasi Chatbot Berbasis Android.”

[3] “Personalized Chatbot Optimization Using Open AI.”

[4] “AI Chatbot Using the MERN Stack and OpenAI API.”

[5] “Voice Command Chatbot with ChatGPT Integration.”

[6] “MindMate: An AI-Powered Chatbot for Mental Health Support.”

[7] “GENIE: Intelligent Chatbot.”

[8] “Selection of Large Language Model for Development of Interactive Chat Bot for SaaS Solutions.”

[9] “Cooking and Food Information Chatbot System Using GPT-3.”

[10] “Thinking Assistants: LLM-Based Conversational Assistants.”

[11] Google AI for Developers, “Gemini API Documentation,” [Online]. Available: https://ai.google.dev/gemini-api/docs. [Accessed: 2026].

[12] Google, “NotebookLM,” [Online]. Available: https://notebooklm.google. [Accessed: 2026].